SIONIGDHA SADHUKHAN
@snigdha.lee75@gmail.com

DATA STRUCTURES AND ALGO SOLUTIONS

Exercise 2: E-commerce Platform Search Function

Scenario:

You are working on the search functionality of an e-commerce platform. The search needs to be optimized for fast performance.

Steps:

1. Understand Asymptotic Notation:
    - Explain Big O notation and how it helps in analyzing algorithms.
    - Describe the best, average, and worst-case scenarios for search operations.
2. Setup:
    - Create a class Product with attributes for searching, such as productId, productName, and category.
3. Implementation:
    - Implement linear search and binary search algorithms.
    - Store products in an array for linear search and a sorted array for binary search.
4. Analysis:
    - Compare the time complexity of linear and binary search algorithms.
    - Discuss which algorithm is more suitable for your platform and why.

Soln:

***1**. Big O Notation: Measures how execution time grows as input size increases for algorithm.*

O(1) -> Constant time
O(log n) -> Logarithmic time
O(n) -> Linear time
O(n²) -> Quadratic time

Best Case: Big(OMEGA)
Algorithm takes atleast this amount of time (lower bound)

Average Case:Big (THETA)
Element found somewhere in the middle.(Tight bound)

Worst Case:Big (O)
Element found at the end or not present.the algo rithm cannot take time more than this to finish.(this takes usually the highest time execution case)

2. class Product {
int productId;
String productName;
String category;
Product(int productId, String productName, String category) {
this.productId = productId;
this.productName = productName;
this.category = category;

```
    }
}
```

3.

LINEAR SEARCH

```
class Search {
static Product linearSearch(Product[] products, int id) {

    for(Product p : products) {
        if(p.productId == id) {
        return p;
        }
   }
   return null;
}
```

BINARY SEARCH

```
static Product binarySearch(Product[] products, int id) {
int low = 0;
int high = products.length - 1;
while(low <= high) {
    int mid = (low + high) / 2;
    if(products[mid].productId == id)
    return products[mid];
    else if(products[mid].productId < id)
    low = mid + 1;
    else
        high = mid - 1;
    }
    return null;
}
```

4.

time complexity of linear search: O(n)

Best Case: O(1)

Average Case: O(n)

Worst Case: O(n)

time complexity of binary search : O(log n)

Best Case: O(1)

Average Case: O(log n)

Worst Case: O(log n)

Binary Search is better for this e-commerce platform.

Reason:

Linear Search
100000 products
May check 100000 items

Binary Search

100000 products
About 17 comparisons only

Advantages of Binary Search:

Faster searching ,Better performance for large datasets ,Scalable for e-commerce applications

Exercise 7: Financial Forecasting

Scenario:

You are developing a financial forecasting tool that predicts future values based on past data.

Steps:

1. **Understand Recursive Algorithms:**
   - Explain the concept of recursion and how it can simplify certain problems.
2. **Setup:**
   - Create a method to calculate the future value using a recursive approach.
3. **Implementation:**
   - Implement a recursive algorithm to predict future values based on past growth rates.
4. **Analysis:**
   - Discuss the time complexity of your recursive algorithm.
   - Explain how to optimize the recursive solution to avoid excessive computation.

SOLN:

1.

Recursion is the phenomenon when a method or function calls itself , till a base condition is met.

A function which calls itself is a recursive function.

2.

```
class FinancialForecast {
```

3.

```
 static double futureValue(double currentValue, double growthRate,int years) {
       if (years == 0)
       return currentValue;
  return futureValue(currentValue * (1 + growthRate),growthRate,years - 1);
   }
```

4.

```
public static void main(String[] args) {
  double currentValue = 10000;
  double growthRate = 0.10; // 10%
  int years = 5;
  double result =futureValue(currentValue,growthRate,years);
       System.out.println("Future Value = " + result);
  }

}
```